Part I: On Device Data Collection and Motion Classification

Edge Impulse URL: `https://studio.edgeimpulse.com/public/1080776/live`

```
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 164 ms, inference 532 us, anomaly 0 ms, postprocessing 68 us
#Classification predictions:
  Idle: 0.972656
  Lift: 0.027344
```

Figure 1: Idle Serial Output

```
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 163 ms, inference 540 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
  Idle: 0.000000
  Lift: 0.996094
```

Figure 2: Lift Serial Output

Part II: Tiny Ensemble Learning

Question 1: Model architecture and ensemble flow

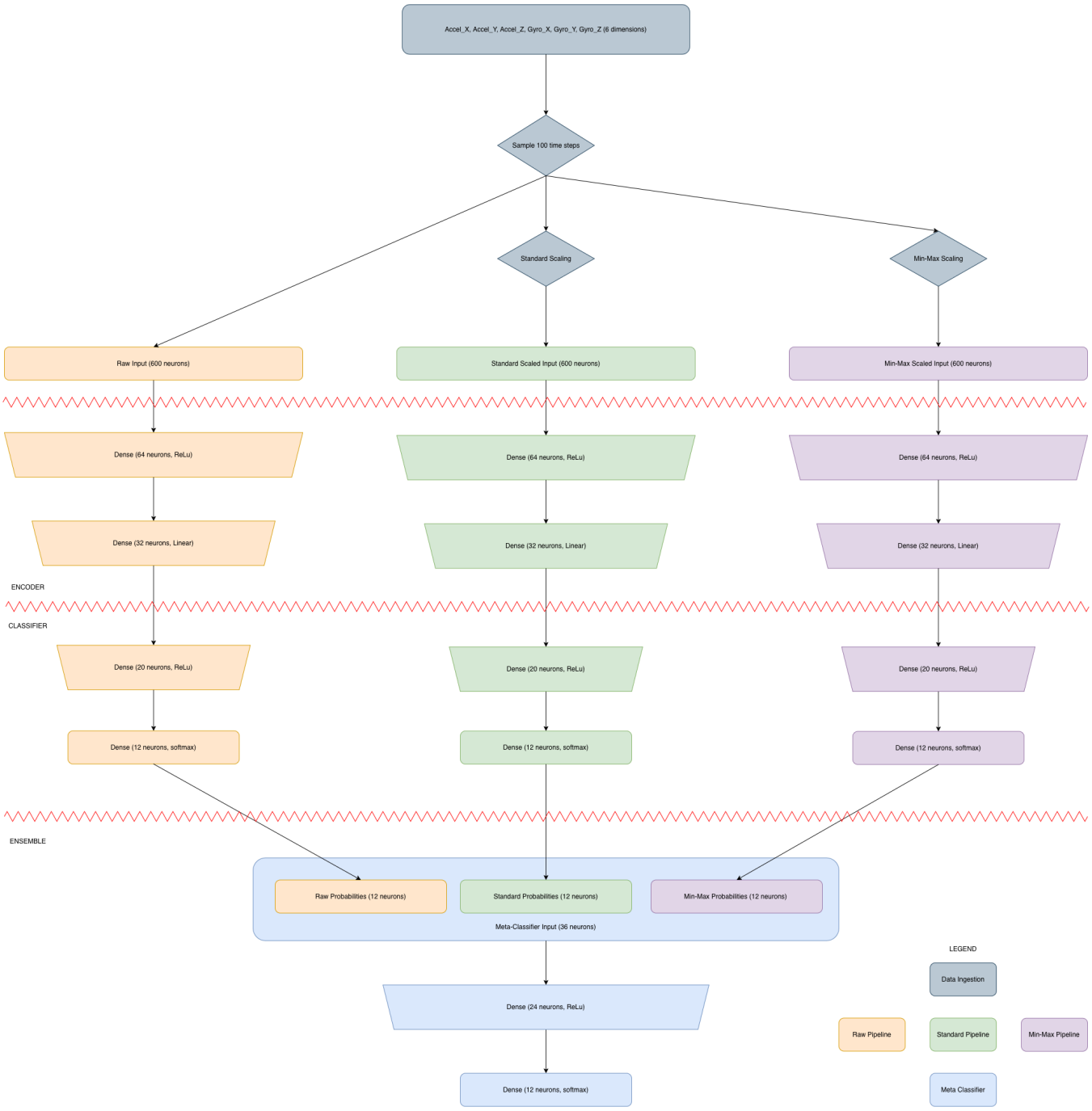

Figure 3: Ensemble Classifier Architecture

Question 2: Pruning and QAT Methodology

We prune and quantize the three encoder-classifier model branches and the meta classifier models individually. All four models are pruned with a Polynomial Decay pruning schedule with an initial sparsity of 0.10 and a final sparsity of 0.90 with 500 pruning steps. The pruned model is then fine tuned on the training data for 5 epochs.
We first strip the model after pruning because the unstripped model consists of all weights and their corresponding masks. We need to apply these masks to ensure that only the pruned neurons have non-zero values. In addition, we need to employ a unique `MaskEnforcerCallback` when we apply Quantization Aware Training to the stripped pruned model. We call `extract_dense_weight_masks` on the stripped pruned model to extract the original weights and the masks on the non-zero weights from the stripped model. We need these dense masks because weights for neurons that were originally pruned out could be assigned a non-zero value when we apply QAT. We have to apply `MaskEnforcerCallback` to ensure that neurons that were pruned out remain pruned out during and after QAT. We apply QAT for 5 epochs for further fine-tuning.
Finally, this model is quantized into an INT8 model using linear quantization. A representative dataset is necessary in order to determine ranges of possible values for weights. Knowing these ranges is necessary for linear quantization. Pruning helps us eliminate weights whose values are close to zero from our models. These weights end up taking memory while not contributing much predictive value. Close to 80 percent of the weights in all four models ended up being unused. INT8 quantization helps reduce the memory footprint even further because each weight would take up 8 bits instead of 32 bits in float32. Those extra bits represent further decimal places which also do not contribute significant predictive value overall. At the end, all these four models combined take up about 137.3kB in memory which is crucial if we want to deploy all these models onto an Arduino board with only 256kB of memory.

Question 3: Arduino deployment and real-world behavior

```
Final prediction: Lying down
Class index: 2
Confidence score: 0.8398
----------------------------------------
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0039, 0.8125, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0195, 0.1641
Standard-scaled model scores: 0.0000, 0.0000, 0.9570, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0430
Min-max-scaled model scores: 0.0352, 0.1055, 0.0391, 0.0000, 0.2266, 0.0000, 0.3398, 0.0000, 0.0039, 0.0234, 0.0000, 0.2266
Meta-classifier scores: 0.0352, 0.0039, 0.8398, 0.0000, 0.0781, 0.0039, 0.0156, 0.0000, 0.0000, 0.0000, 0.0078, 0.0078
----------------------------------------
Final prediction: Lying down
Class index: 2
Confidence score: 0.8398
----------------------------------------
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0039, 0.8125, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0195, 0.1641
Standard-scaled model scores: 0.0000, 0.0000, 0.9570, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0430
Min-max-scaled model scores: 0.0352, 0.1055, 0.0391, 0.0000, 0.2266, 0.0000, 0.3398, 0.0000, 0.0039, 0.0234, 0.0000, 0.2266
Meta-classifier scores: 0.0352, 0.0039, 0.8398, 0.0000, 0.0781, 0.0039, 0.0156, 0.0000, 0.0000, 0.0000, 0.0078, 0.0078
----------------------------------------
Final prediction: Lying down
Class index: 2
Confidence score: 0.8398
----------------------------------------
```

Figure 4: Arduino Serial Output for Deployed Ensemble Model

Most of the time, the model predicted the "Lying down" state which is acceptable for a board at rest. I did notice the model predict "Lying down" when I moved the board around with lower confidence scores. There could be mismatches between the axes of the training dataset and the Arduino board. Adding real measurements taken from the Arduino board for each of the 12 possible states could improve reliability while paying close attention to the original orientation of the measuring device that collected the training data.

Question 4: Deployment Behavior on the Arduino

I noticed unexpected predictions when I moved the board around. I also saw the model switch between "Climbing stairs" and "Jump front and back". It's possible the raw measurements may have different units and orientations between the Arduino board and the original measuring device. This could corrupt the ensemble model. Assuming that this data was collected from a device attached to a left ankle bracelet, it's also possible that walking sideways in the Arduino's frame of reference can mean walking forward in the original measurement device's frame of reference.